

SWE1 Level OTA Project Documentation

Over-The-Air (OTA) Software Update System

Document Version: 1.0

Prepared By: SWE1 Engineer

Project Type: Automotive OTA System

Purpose: Reference document for Retrieval-Augmented Generation (RAG) project

1. Introduction

1.1 Purpose

This document describes the basic architecture, components, workflow, and implementation details of an Over-The-Air (OTA) software update system at a Software Engineer 1 (SWE1) level. This document is intended for:

- New engineers joining OTA team
- Documentation for RAG-based AI assistant
- Basic understanding of OTA system architecture

1.2 Scope

This document covers:

- OTA system overview
- Components of OTA system
- Basic OTA workflow
- Client side implementation
- Server side overview
- Error handling
- Security considerations

1.3 Definitions

Term	Description
OTA	Over-the-Air software update
ECU	Electronic Control Unit
TCU	Telematics Control Unit

Term	Description
Bootloader	Software responsible for firmware update
Firmware	Software running on ECU
Manifest	Metadata describing update package
Delta Update	Update containing only changed portions

2. System Overview

OTA system allows remote software update of vehicle ECUs without requiring physical service visits.

2.1 High Level Architecture

OTA system consists of the following components:

1. OTA Cloud Server
2. Vehicle TCU
3. Target ECU
4. Update Storage
5. Communication Layer

2.2 OTA Architecture Diagram

Cloud Server

|

|

Vehicle TCU

|

|

Target ECU

3. OTA System Components

3.1 OTA Cloud Server

Responsibilities:

- Store firmware images

- Maintain update metadata
- Authenticate vehicles
- Manage update campaigns
- Provide download URLs

Main Modules:

- Update Manager
- Campaign Manager
- Authentication Service
- Storage Service

3.2 Vehicle TCU

Responsibilities:

- Check for updates
- Download update package
- Verify update
- Install update
- Report status

Main Modules:

- OTA Client
- Download Manager
- Update Manager
- Report Manager

3.3 Target ECU

Responsibilities:

- Receive firmware
- Validate firmware
- Install firmware
- Report installation status

3.4 Bootloader

Responsibilities:

- Receive firmware
 - Validate signature
 - Flash firmware
 - Rollback if failure
-

4. OTA Workflow

4.1 Basic OTA Flow

Step 1: Vehicle checks for update

Step 2: Server responds with metadata

Step 3: Vehicle downloads firmware

Step 4: Vehicle verifies firmware

Step 5: Vehicle installs firmware

Step 6: Vehicle reports status

4.2 Detailed OTA Flow

1. Vehicle boots
 2. OTA client starts
 3. Client sends request to server
 4. Server checks campaign
 5. Server sends manifest
 6. Client downloads package
 7. Client verifies checksum
 8. Client sends firmware to ECU
 9. ECU installs firmware
 10. ECU reboots
 11. Vehicle reports success
-

5. Client Side Implementation

5.1 OTA Client Modules

5.1.1 Update Checker

Responsibilities:

- Periodic update check
- Trigger manual update

Example Pseudocode:

```
check_for_update()
{
    send_request_to_server();
    if(update_available)
    {
        start_download();
    }
}
```

5.1.2 Download Manager

Responsibilities:

- Download firmware
- Resume download
- Store locally

Example Pseudocode:

```
download_package()
{
    open_connection();
    download_chunks();
    verify_download();
}
```

5.1.3 Verification Module

Responsibilities:

- Check checksum

- Verify signature

Example Pseudocode:

```
verify_package()  
{  
    calculate_checksum();  
    compare_checksum();  
}
```

5.1.4 Installation Manager

Responsibilities:

- Send firmware to ECU
- Monitor installation

6. Server Side Overview

6.1 API Endpoints

Endpoint	Method	Description
/check_update	POST	Check update
/download	GET	Download firmware
/report	POST	Report status

6.2 Example Manifest

```
{  
    "version": "1.2.0",  
    "ecu": "ADAS",  
    "size": 102400,  
    "checksum": "abc123",  
    "url": "https://ota-server/update.bin"  
}
```

7. Error Handling

Common Errors:

Error	Description
Download failure	Network issue
Verification failure	Corrupted file
Installation failure	ECU error

Retry Strategy:

- Retry download 3 times
 - Resume download
 - Rollback on failure
-

8. Security Considerations

Security Features:

- Secure boot
- Firmware signing
- TLS communication
- Authentication

Security Flow:

1. Verify server certificate
 2. Download signed firmware
 3. Verify signature
 4. Install firmware
-

9. Logging

Logs Required:

- Update check log
- Download log

- Installation log
- Error log

Example Log:

[INFO] Checking update

[INFO] Update available

[INFO] Download started

[INFO] Download completed

[INFO] Installation successful

10. Testing

Testing Types:

- Unit testing
- Integration testing
- System testing
- Regression testing

Test Cases:

Test Case	Description
Update success	Normal flow
Download failure	Network off
Power loss	During install

11. Future Enhancements

- Delta update support
 - Multi ECU update
 - Parallel download
 - Background update
-

12. Folder Structure

ota_project

|

|-- client

|-- server

|-- bootloader

|-- docs

|-- test

13. Summary

This document provides a basic overview of OTA system architecture and workflow. This document is intended for SWE1 level understanding and for use in RAG-based AI systems.

14. Keywords for RAG

OTA

Firmware Update

ECU

TCU

Bootloader

Manifest

Download Manager

Verification

Installation

Automotive OTA

End of Document